SOFTWARE-ONTWERP · FINALE BEOORDELINGSVERSIE

Software-ontwerp ZorgApp 1.3

Een grondig ontwerpdocument van de Unity-game, de .NET Web API, de koppeling met Microsoft SQL Server en de GitHub Actions CI/CD-pipeline. Deze versie is expliciet opgezet om hoog te scoren op visuele helderheid, technische onderbouwing en architectuurkwaliteit.

Frontend
Unity 6.0.36f1

Backend
ASP.NET Core Web API (.NET 9)

Opslag
SQL Server + Dapper

CI/CD
GitHub Actions → Azure

1. Inleiding

De ZorgApp bestaat uit twee repositories die samen één functioneel systeem vormen. De repo **avansbitbybit/zorgapp1.3** bevat de Unity-client, waarin de gebruiker kan inloggen, door scènes navigeren, een avatar aanpassen, dagboekitems bekijken en afspraken beheren. De repo **avansbitbybit/zorgapp1.3webapi** bevat de server-side logica: authenticatie, autorisatie, businessregels en databasecommunicatie.

Voor dit document zijn beide codebases opnieuw zorgvuldig onderzocht, inclusief scene-configuratie, API-clients, repositories, modellen,

Waarom deze versie sterker is

- Gebaseerd op echte code uit beide repositories
- Bevat visuele architectuur-, UML-, ERD- en CI/CD-diagrammen
- Legt ontwerpkeuzes expliciet uit
- Koppelt die keuzes aan onderhoudbaarheid, schaalbaarheid en performance

21 actieve scenes

4 API-domeinen

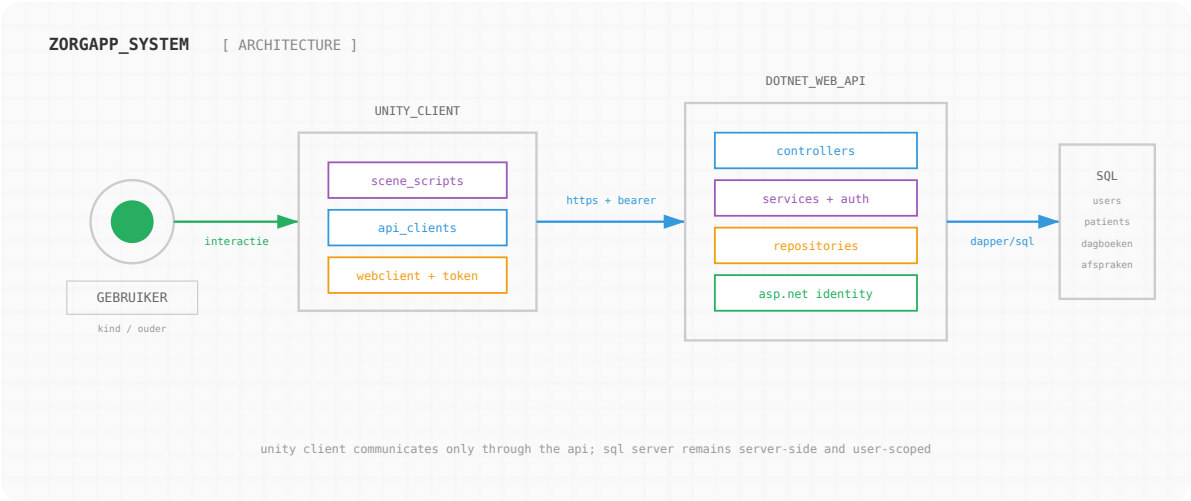
4 kern-diagrammen

testprojecten en de GitHub Actions workflow. Het doel is daarom niet alleen een beschrijving van mappen en klassen, maar een echte architectuuranalyse van de manier waarop de onderdelen samenwerken.

2. Architectuurprincipe

Het systeem volgt primair een **client-serverarchitectuur**. De Unity-app is verantwoordelijk voor de presentatie, gebruikersinteractie en scènebesturing. De Web API is verantwoordelijk voor authenticatie, autorisatie, gegevensverwerking en persistente opslag. Binnen de backend is daarnaast een **gelaagde architectuur** zichtbaar: controllers, services, repositories en modellen hebben elk een eigen verantwoordelijkheid.

- **Client-server:** Unity communiceert via HTTP met de API.
- **Separation of concerns:** UI-logica, API-communicatie en persistence zijn gescheiden.
- **User-scoped design:** data wordt server-side gefilterd op `UserId`.
- **Deployable backend:** de API kan via een pipeline automatisch worden gebouwd, getest en gedeployed.



Figuur 1 — Hoofdarchitectuur van het systeem, gemaakt in een technische diagramstijl gericht op documentatie.

> **Belangrijk ontwerpprincipe:** de Unity-client praat nooit direct met SQL Server. Alle toegang loopt via de Web API. Daardoor blijven beveiliging, validatie en gebruikersafscherming gecentraliseerd op de server.

3. Frontendanalyse: Unity-game

De Unity-repo draait op **Unity 6.0.36f1** en bevat 21 actieve scènes. In `EditorBuildSettings.asset` zijn onder andere *IntroductieScene*, *LoginScene*, *Behandeltraject*, *DagboekScene*, *Accountgegevens*, *MenuScene*, *Charactercreator* en meerdere missie-scenes geactiveerd. Hierdoor is het systeem duidelijk opgebouwd als een scene-gedreven game/applicatie.

Laag	Beschrijving	Voorbeelden
Scene/UI-laag	Verwerkt knoppen, navigatie en scherminteractie	`LoginScript`, `MenuSceneManager`, `PollSceneManager`
Applicatielaag	Beheert specifieke gebruikersflow per scene	`DagboekSceneManager`, `AfspraakManager`, `Avatarcostumizer`
Integratielaag	Vertaling van game-acties naar API-calls	`UserApiClient`, `PatientApiClient`, `DagboekApiClient`, `AfspraakApiClient`
Transportlaag	HTTP requests, tokenbeheer en foutafhandeling	`WebClient`

Een belangrijke bevinding is dat de API-base URL op meerdere plekken terugkomt. In de Unity-scenes en prefabs werden onder andere `https://avansict22257801.azurewebsites.net` en `https://avansict2233343.azurewebsites.net` aangetroffen. Dat werkt functioneel, maar het maakt configuratie minder centraal beheersbaar.

Belangrijke ontwerpkeuzes aan de clientzijde

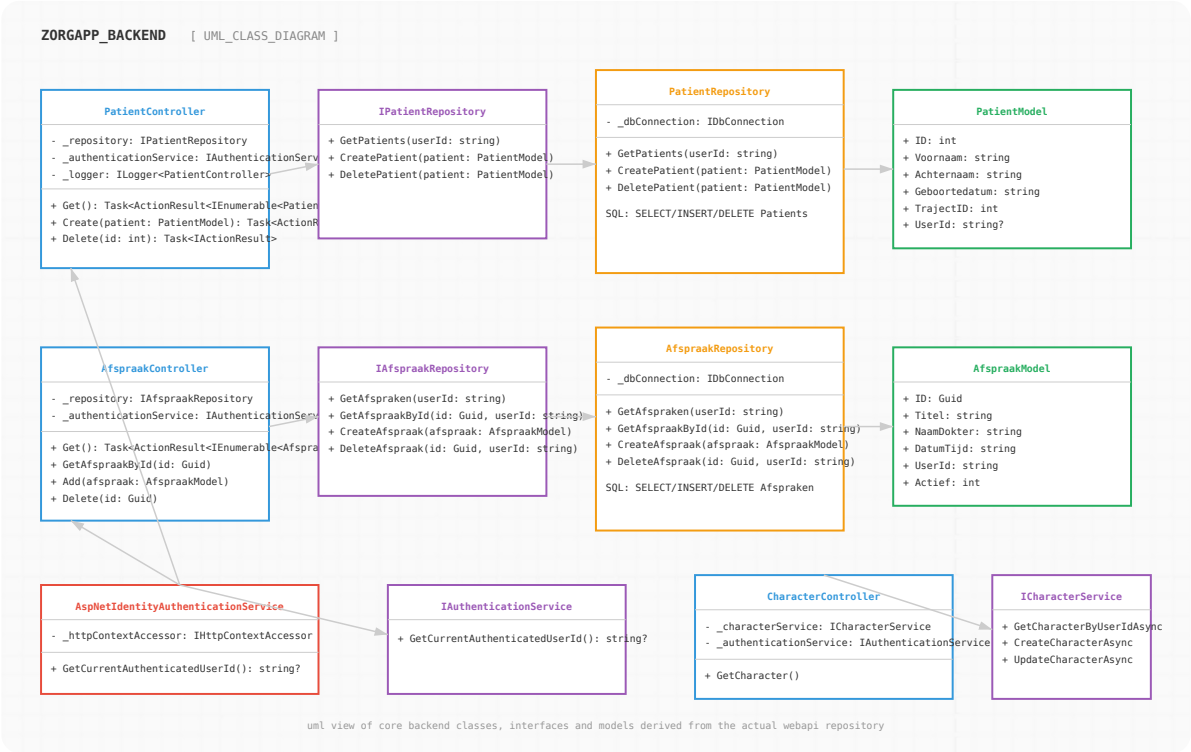
- **Tokenopslag in PlayerPrefs:** praktisch voor een schoolproject, maar beperkt veilig voor productie.
- **Centrale WebClient:** voorkomt duplicatie van requestlogica en houdt API-calls consistenter.
- **API-clients per domein:** verbeteren leesbaarheid en scheiden scene code van HTTP-transport.

4. Backendanalyse: .NET Web API

De webapi-repo gebruikt **.NET 9**, **ASP.NET Core Web API**, **Dapper** en **Avans.Identity.DapperOrm**. In de solution zitten twee projecten: `ZorgWebApi` voor de runtime-applicatie en `UnitTestenZorgApp` voor de tests. In `Program.cs` worden controllers, Swagger/OpenAPI, repositories, services, Dapper stores en Identity API endpoints geregistreerd.

De dominante backend-onderdelen zijn:

- **Controllers:** `PatientController`, `DagboekController`, `AfspraakController`, `CharacterController`
- **Services:** `CharacterService`, `AspNetIdentityAuthenticationService`
- **Repositories:** voor Patients, Dagboeken, Afspraken en Characters
- **Authenticatie:** ASP.NET Identity API endpoints onder `/account`



Figuur 2 — UML class diagram van kernklassen, interfaces en modellen in de backend. Dit diagram laat ook types en afhankelijkheden zien.

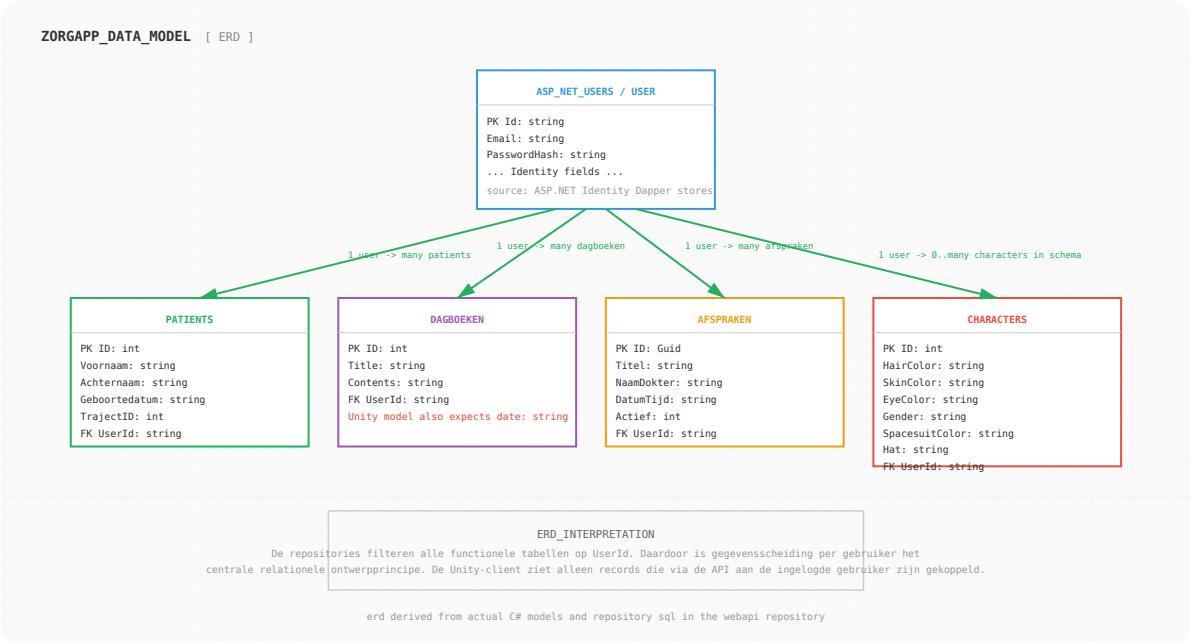
> **Belangrijke codebevindingen:** in `DagboekRepository.UpdateDagboek` zit een SQL-syntaxfout door een extra komma vóór `WHERE`. Daarnaast worden `IAfspraakRepository` en `MapControllers()` dubbel geregistreerd in `Program.cs`. Dit zijn waardevolle observaties voor een hoog scorend ontwerp, omdat ze direct laten zien waar ontwerp en implementatie nog strakker kunnen worden gemaakt.

5. Relatie tussen API en Microsoft SQL Server

De API maakt in `Program.cs` een `SqlConnection` aan op basis van een connection string uit configuratie. Vervolgens gebruiken de repositories Dapper-query's voor CRUD-operaties. Voorbeelden uit de code zijn onder andere:

- `SELECT * FROM Patients WHERE UserId = @UserId`
- `SELECT * FROM Dagboeken WHERE UserId = @UserId`
- `SELECT * FROM Afspraken WHERE UserId = @UserId`
- `SELECT * FROM Characters WHERE UserId = @UserId`

Daardoor is de database logisch opgezet rondom de gebruiker. Niet de Unity-client, maar de backend bepaalt welke gegevens een gebruiker mag zien. Dit is een belangrijk ontwerpvoordeel op het gebied van beveiliging en onderhoudbaarheid.



Figuur 3 — ERD van het functionele datamodel. Alle hoofdtabellen koppelen terug naar de gebruiker via `UserId`.

De belangrijkste functionele entiteiten zijn:

- **Patients** — patiëntgegevens per gebruiker
- **Dagboeken** — dagboekitems per gebruiker
- **Afspraken** — afspraken met dokter, datum/tijd en actieve status
- **Characters** — avatar/personalisatiegegevens

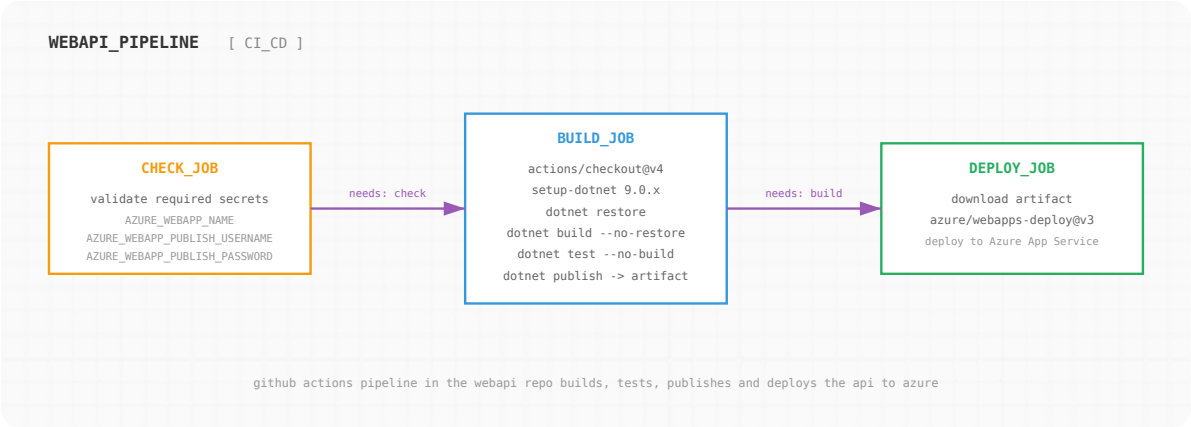
6. Belangrijkste runtime-interacties

De sterkste functionele flow in het systeem is de keten **registreren/inloggen → token ontvangen → token opslaan → Bearer-token meesturen → user-scoped databasequery**. Dit patroon komt terug bij onder meer het ophalen van patiënten, dagboekitems en afspraken.

Stap	Wat gebeurt er	Componenten
1	Gebruiker logt in via Unity	`LoginScript` → `UserApiClient`
2	Unity stuurt POST naar `/account/login`	`WebClient` → Web API
3	API valideert via ASP.NET Identity	Identity + SQL Server
4	Unity ontvangt token en slaat dit op	`PlayerPrefs`
5	Vervolgrequests bevatten Bearer-token	`WebClient`
6	API filtert records op `UserId`	controllers + repositories + SQL

7. CI/CD-pipeline

In de webapi-repo is een GitHub Actions workflow aanwezig: `.github/workflows/github_workflow_simplified (2).yml`. Deze pipeline wordt gestart op pushes naar `main`, `deployment` en `master`, en kan ook handmatig gestart worden via `workflow_dispatch`.



Figuur 4 — CI/CD-flow voor de Web API: secrets-check, build/test/publish en deployment naar Azure App Service.

De pipeline bestaat uit drie fasen:

- **check:** controleert of verplichte Azure-secrets aanwezig zijn
- **build:** voert `dotnet restore`, `dotnet build`, `dotnet test` en `dotnet publish` uit
- **deploy:** downloadt het artifact en probeert naar Azure App Service te deployen via `azure/webapps-deploy@v3`

Ontwerpobservatie bij de pipeline: inhoudelijk is de pipeline sterk genoeg voor een goede beoordeling, maar technisch niet perfect strak. In de deploy-stap wordt `env.AZURE_WEBAPP_NAME` gebruikt terwijl bovenin vooral secrets worden gecontroleerd. Dat is precies het soort detail dat je in een excellent ontwerp benoemt: niet alleen de architectuur op hoofdlijnen, maar ook de uitvoerbaarheid van de deploymentketen.

8. Sterke punten, risico's en alternatieven

Sterke punten

- Duidelijke client-serverarchitectuur
- Goede basis van gelaagde backendarchitectuur
- User-scoped filtering via `UserId`
- Snelle en transparante data-access via Dapper
- Aanwezige testprojecten en CI/CD-pipeline

Gevonden risico's

- Tokenopslag in `PlayerPrefs` is beperkt veilig
- Frontend en backend lijken niet op alle endpoints volledig synchroon
- Configuratie van base URLs is versnipperd
- Niet alle businessregels zitten in services
- Losse implementatiefouten kunnen door handgeschreven SQL sneller insluipen

Mogelijke alternatieven

- **Entity Framework Core** in plaats van Dapper — meer abstractie en migraties, maar minder directe SQL-controle.
- **Secure token storage** in plaats van PlayerPrefs — veiliger, maar complexer in Unity.
- **Central app config** voor API-base URLs — onderhoudsvriendelijker dan scene-specifieke configuratie.
- **Uitbreiding van servicelaag** — verplaatst businessregels uit controllers naar een beter testbare laag.

9. Koppeling aan de rubric

Rubric-eis	Hoe deze documentatie eraan voldoet
Overzicht van componenten en interacties	De frontend, backend, SQL Server en pipeline zijn expliciet beschreven en visueel gemaakt.
Architectuurprincipes toepassen	Client-server, gelaagde architectuur en separation of concerns zijn inhoudelijk uitgelegd en gekoppeld aan echte code.
Visuele weergave van componenten	Architectuurdiagram, UML-diagram, ERD en CI/CD-diagram zijn toegevoegd.
Ontwerpkeuzes en alternatieven	Dapper, Identity, Unity, tokenopslag en configuratiekeuzes zijn uitgelegd met alternatieven.
Schaalbaarheid / onderhoudbaarheid / performance	Elke keuze is gekoppeld aan impact op beheer, veiligheid en verdere uitbreiding.

Beoordelingsinschatting: deze versie is duidelijk gericht op **goed tot excellent**, omdat het niet alleen beschrijft wat er gebouwd is, maar ook waarom het zo is opgebouwd, hoe de onderdelen samenwerken en waar de belangrijkste ontwerpafwegingen en verbeterpunten zitten.

10. Conclusie

De ZorgApp 1.3 gebruikt een architectuur die technisch logisch en goed verdedigbaar is voor dit project. De Unity-client verzorgt de interactieve gebruikerservaring, de .NET Web API centraliseert authenticatie en businesslogica, en SQL Server bewaart zowel identity-data als functionele appdata. Dankzij de aanwezigheid van repositories, interfaces, testprojecten en CI/CD is er meer aanwezig dan alleen een werkend prototype: er is een duidelijke basis voor verdere professionele doorontwikkeling.

Het ontwerp is vooral sterk omdat de relatie tussen componenten werkelijk zichtbaar is in de code: Unity gebruikt API-clients en een centrale WebClient, de API gebruikt Identity en Dapper, en de databasekoppeling draait volledig op server-side filtering met `UserId`. Daardoor is het software-ontwerp niet alleen compleet genoeg om de applicatie te realiseren, maar ook rijk genoeg gedocumenteerd om hoog te scoren binnen de rubric.

Gebaseerd op de repositories `avansbitbybit/zorgapp1.3` en `avansbitbybit/zorgapp1.3webapi`, inclusief code-analyse, scene-configuratie, modellen, repositories, tests en GitHub Actions workflow.